

10. Fonctions CM / TD (hors TP)

Fonctions vues dans les TD2-TD6 et cours magistraux, susceptibles de tomber à l'examen.

parse_aux + parse_expr — version TD2 avec const char**

Différence clé par rapport à la version TP : utilise const char** (pointeur sur pointeur) au lieu de int *pos.

Avec char* : chaque appel reçoit une COPIE du pointeur → impossible de partager le curseur.

Avec char** : tous les appels voient et modifient le MÊME pointeur. **s = lire, (*s)++ = avancer.

parse_aux(const char **s)

Helper récursif. **s lit le caractère courant, (*s)++ avance le curseur partagé. Pas d'espaces dans la chaîne.

```
static link parse_aux(const char **s) {
    char c = **s; /* lire le caractere courant */
    (*s)++; /* avancer le curseur partage */
    if (c=='+' || c=='-' || c=='*' || c=='/') {
        link g = parse_aux(s);
        link d = parse_aux(s);
        return cons_binary_tree(c, g, d);
    } else {
        return cons_binary_tree(c, NULL, NULL); /* feuille */
    }
}
```

parse_expr(const char *s)

Point d'entrée : prend const char*, passe son adresse à parse_aux.

```
link parse_expr(const char *s) {
    return parse_aux(&s);
}
```

max_binary_tree(const link bt) — TD1

Retourne le maximum des étiquettes. Requier bt non vide. O(n)

```
item max_binary_tree(const link bt) {
    if (is_empty_binary_tree(bt->left) && is_empty_binary_tree(bt->right))
        return bt->label; /* feuille */
    item root_val = get_binary_tree_root(bt);
    item lm = is_empty_binary_tree(bt->left) ? root_val : max_binary_tree(bt->left);
    item rm = is_empty_binary_tree(bt->right) ? root_val : max_binary_tree(bt->right);
    item child_max = less(lm, rm) ? rm : lm;
    return less(root_val, child_max) ? child_max : root_val;
}
```

count_leaves_binary_tree(const link bt) — TD1

Compte les feuilles (noeuds sans enfants). O(n)

```
int count_leaves_binary_tree(const link bt) {
    if (is_empty_binary_tree(bt)) return 0;
    if (is_empty_binary_tree(bt->left) && is_empty_binary_tree(bt->right))
        return 1;
    return count_leaves_binary_tree(bt->left)
        + count_leaves_binary_tree(bt->right);
}
```

sum_binary_tree(const link bt) — TD / partiel

Somme de toutes les étiquettes (item = int). $O(n)$

```
int sum_binary_tree(const link bt) {
    if (is_empty_binary_tree(bt)) return 0;
    return bt->label
    + sum_binary_tree(bt->left)
    + sum_binary_tree(bt->right);
}
```

infix_RB + tri_RB — TD5/TD6

Tri par LLRBT. Phase 1 : n insertions $O(\log n)$. Phase 2 : parcours infixe $O(n)$. Total $O(n \log n)$.

```
static int infix_RB(llrbt_link h, item *a, int i) {
    if (h==NULL) return i;
    i = infix_RB(h->left, a, i);
    a[i++] = h->label;
    i = infix_RB(h->right, a, i);
    return i;
}

void tri_RB(item *a, int n) {
    llrbt_link root = NULL;
    /* Phase 1 : construction du LLRBT */
    for (int i=0; i<n; i++)
        root = insert_LLRT(root, a[i]);
    if (root!=NULL) root->color = BLACK;
    /* Phase 2 : lecture infixe -> tableau trie */
    infix_RB(root, a, 0);
    delete_llrbt(&root);
}
```